# HarmonyOS Co-Devl Guidebook

| | |
|---|---|
| Date Created | April 8, 2026 11:41 AM |

## Harmony Operator Runbook

> **Purpose**: Human-facing SOP for pushing Harmony-native development forward without contaminating shared truth or the Android lineage. **Status**: Active operator runbook **Date**: 2026-04-08 **Primary Laws**:
>
> - `docs/platforms/harmony/native-development-framework.md`
> - `docs/sops/tracker-governance.md`
> - `docs/specs/platform-governance.md` **Working Tracker**:
> - `docs/plans/harmony-tracker.md` **Related Docs**:
> - `docs/plans/tracker.md`
> - `docs/platforms/harmony/tingwu-container.md`
> - `docs/cerb/interface-map.md`

---

### 1. Operator Role

The human operator is not the person who blindly pushes code work forward.

The operator owns:

- classifying the slice before coding starts
- fixing the capability boundary before coding starts
- routing the slice to the correct tracker and docs
- briefing the agent with the right truth sources
- reviewing evidence before the slice is allowed to advance

The operator does **not** treat Harmony as a second product-truth owner.

Current repo posture:

- Android remains the main live behavior reference
- Harmony is translation-first and doc-first native rewriting
- the current active Harmony lane is a bounded transient container, not a parity program

---

## 2. The Operator Flow

Run every Harmony slice in this order.

### Step 1: Classify the lane

Choose one:

- `shared truth`
- `harmony-native delivery`
- `governance`

If the work changes product meaning, user promise, or shared acceptance criteria, it is **not** only Harmony delivery.

### Step 2: Classify the Harmony slice

For Harmony-native delivery, choose one:

- `translation`
- `curated-container`
- `divergence`

Use this rule:

- same intent, same contract, same promise → `translation`
- reduced supported capability set → `curated-container`
- real platform-owned behavior difference → `divergence`

### Step 3: Freeze the capability boundary

Before any implementation, write down:

- supported capability
- disabled capability

- user-visible limitation
- what must fail closed instead of pretending parity

If this cannot be stated clearly, the slice is not ready to build.

## Step 4: Read truth in the correct order

Use this reading order:

1. shared docs and contracts
2. core-flow docs when present
3. Cerb `spec.md`
4. Cerb `interface.md`
5. Android live behavior
6. Harmony overlay docs

If Android code is the only clear behavior evidence, extract the behavior into docs before treating the Harmony answer as settled.

## Step 5: Route the slice

Current default:

- use `docs/plans/harmony-tracker.md` as the primary standing tracker for Harmony slices
- use `docs/plans/tracker.md` only for portfolio, branch, and governance summary
- create an execution brief only when one approved slice needs bounded execution detail

## Step 6: Brief the agent

A good Harmony brief must include:

- feature intent
- slice class (`translation`, `curated-container`, or `divergence`)
- supported set
- disabled set
- source docs

- required evidence
- contamination boundary

Recommended brief shape:

```
Build the Harmony-native version of [feature] from the shar
ed spec.
This slice is [translation / curated-container / divergenc
e].
Supported capability: [...]
Disabled capability: [...]
Truth sources: [...]
Do not port Compose structure. Rewrite natively in ArkTS/Ar
kUI.
Return doc sync plus evidence for UI translation and backen
d/dataflow if touched.
```

## Step 7: Review evidence before advancing

Before you move the tracker state forward, confirm:

- docs were updated in the same session
- supported and disabled capability are still honest
- no unsupported flow is left half-wired
- backend/dataflow evidence exists when protocol, storage, runtime, or payload seams changed
- CLI verification exists when build/device/runtime behavior was part of the slice
- Harmony-native files stayed inside the Harmony-owned root

## Step 8: Decide the outcome

Use only these outcomes:

- `advance`: docs and evidence are aligned enough to move the slice forward
- `hold`: truth is still unclear, evidence is missing, or capability boundary is not honest

- `reject`: the slice fakes parity, mixes ownership, or contaminates the Android tree

---

## 3. What Evidence the Operator Should Ask For

Ask for evidence by slice type.

### UI translation / native rewrite

Require:

- the source behavior being translated
- what was rewritten natively
- what was intentionally hidden or blocked
- screenshots or equivalent UI proof when visual behavior matters

### Backend / dataflow rewrite

Require:

- the shared contract being preserved
- the old Android-side behavior reference
- the Harmony-native mapping
- proof of runtime, protocol, storage, or payload behavior

### Toolchain / runtime slice

Require:

- build command used
- deploy or device command used when relevant
- log or runtime inspection command used when relevant
- clear note about what remains unverified

Operator rule:

- no “looks right” acceptance for runtime-sensitive Harmony work
- if the slice touched dataflow, require dataflow evidence explicitly

---

# 4. How to Use Cerb `interface.md` for Harmony

Use Cerb docs with this hierarchy:

- `spec.md` owns behavior and ownership
- `interface.md` owns the consumer-facing contract surface
- code shows the current implementation shape

For Harmony, `interface.md` is usually useful as a **semantic translation scaffold**, not as a literal architecture template.

## 4.1 Semantically reusable

These are good reuse targets:

- input and output meaning
- state semantics
- guarantees
- artifact shapes
- consumer do/don't rules

Example:

- `docs/cerb/tingwu-pipeline/interface.md` is largely reusable because it mainly defines long-form audio job meaning, artifact meaning, and degradation rules

## 4.2 Conditionally reusable

These may be reused if they stay platform-neutral:

- repository/service boundaries
- method seams that still fit the Harmony implementation cleanly
- data types that describe shared product meaning rather than Android runtime assumptions

Example:

- `docs/cerb/audio-management/interface.md` is only partially reusable; parts of it describe stable consumer expectations, while parts are tightly shaped around the current Android repository seam

### 4.3 Not directly reusable

These should not be copied forward just because they compile in Android:

- Android lifecycle assumptions
- Android-only hardware delivery paths
- Android runtime types or service boundaries
- method seams that only exist because of the current Android implementation shape

Example:

- `docs/cerb/connectivity-bridge/interface.md` should not be treated as the default native Harmony contract because the interface map already classifies that hardware path as `android-only`

### 4.4 Operator decision rule

When reviewing a Harmony slice, ask:

- is the agent preserving the contract meaning?
- or is the agent copying the Android seam because it already exists?

The correct default is:

- preserve the meaning
- rewrite the seam when the current interface is Android-shaped

---

## 5. Branch and Restore Handling

For the current transient Harmony lane:

- keep branch and restore state visible in `docs/plans/harmony-tracker.md`
- keep portfolio/governance summary in `docs/plans/tracker.md`
- do not let the Harmony branch become a hidden second trunk

Before you treat a Harmony slice as safe to continue from later, confirm:

- branch name is recorded
- purpose is recorded
- baseline commit or tag is recorded

- restore reference is recorded
- current restore confidence is stated honestly

---

## 6. Failure Modes the Operator Should Stop Early

Stop early when you see:

- unsupported Harmony capability hidden behind parity-looking UI
- Android code shape being ported directly as if it were the target architecture
- shared product semantics being changed inside a Harmony-only pass
- missing supported/disabled capability declaration
- missing dataflow evidence for runtime-sensitive rewrite work
- Harmony-native files landing in the Android tree

These are not polish issues. They are control-plane failures.

---

## 7. Acceptance Checklist

Do not advance a Harmony slice unless all are true:

- the slice class is explicit
- supported and disabled capability are explicit
- the right truth sources were used
- the primary tracker is correct
- docs are synced
- required evidence exists
- unsupported capability fails closed
- no Android-tree contamination exists

If any item is missing, hold the slice instead of letting ambiguity accumulate.